Poner a correr el proyecto con Docker

Del `docker compose` hasta tener el juego funcionando en el navegador.

Antes de empezar

Tener Docker Desktop instalado y abierto (el ícono de la ballena debe estar activo). Todos los comandos se ejecutan en una terminal, parada en la carpeta del proyecto, la que contiene `docker-compose.yml`, `Dockerfile` y `manage.py`.

Qué levanta el compose

Servicio	Qué es	Puerto
`api`	El proyecto Django, construido con el Dockerfile. El código de la carpeta se monta en /api.	`8000`
`db`	Postgres 16. Usuario alumno, contraseña ldp, base postgres. Los datos quedan en el volumen db_data.	`5432`

Paso a paso

Docker nombra los contenedores `<carpeta>-<servicio>-1`. Aquí se usan `discretizando-el-tiempo-continuo-api-1` y `discretizando-el-tiempo-continuo-db-1`. Si la carpeta tiene otro nombre, se puede ver el nombre real con `docker ps`.

1. Ir a la carpeta del proyecto

```
cd "ruta/a/discretizando-el-tiempo-continuo"
```

2. Construir la imagen y levantar los contenedores

`--build` reconstruye la imagen de la API; `-d` deja todo corriendo en segundo plano. La primera vez descarga Postgres y las librerías, y puede tardar unos minutos.

```
docker compose up -d --build
```

3. Comprobar que los dos servicios están arriba

Ambos deben aparecer con estado `running`.

```
docker compose ps
```

4. Crear las tablas en la base de datos

Solo hace falta la primera vez, o cuando se agregan migraciones nuevas.

```
docker exec discretizando-el-tiempo-continuo-api-1 python manage.py migrate
```

5. Cargar los personajes y el usuario

Corre `poblar_personajes.py` dentro del contenedor. Crea los personajes con su imagen, deja uno seleccionado y crea el usuario **alumno** con contraseña **usm1234** (con permisos de administrador). Se puede repetir sin duplicar datos.

```
docker exec discretizando-el-tiempo-continuo-api-1 python poblar_personajes.py
```

6. Revisar que la API arrancó sin errores

Debe verse `Starting development server at http://0.0.0.0:8000/`. Con Ctrl+C se sale de los logs sin detener nada.

```
docker logs -f discretizando-el-tiempo-continuo-api-1
```

7. Abrir en el navegador

URL	Para qué sirve
`localhost:8000/api/seleccion-personaje`	Elegir personaje con un click y pasar al tablero.
`localhost:8000/tablero`	El tablero. Se mueve con las teclas w, a, s, d.
`localhost:8000/admin/`	Panel de administración. Usuario **alumno**, contraseña **usm1234**.

Comandos del día a día

Comando	Qué hace
`docker logs -f discretizando-el-tiempo-continuo-api-1`	Ver en vivo lo que imprime la API.
`docker restart discretizando-el-tiempo-continuo-api-1`	Reiniciar solo la API.
`docker exec -it discretizando-el-tiempo-continuo-api-1 bash`	Abrir una terminal dentro del contenedor de la API.
`docker exec -it discretizando-el-tiempo-continuo-db-1 psql -U alumno postgres`	Entrar a la base de datos con psql.
`docker compose down`	Detener y eliminar los contenedores. Los datos de la base se conservan.
`docker compose down -v`	Igual que el anterior, pero borra también los datos de la base.
`docker compose up -d --build`	Volver a levantar todo reconstruyendo (necesario al cambiar requirements.txt o el Dockerfile).

Si algo falla

Síntoma	Causa y solución
`libGL.so.1: cannot open shared object file`	Está instalado `opencv-python`. Usar `opencv-python-headless` en requirements.txt y reconstruir con `--build`.
`password authentication failed for user`	La API usa credenciales distintas a las del servicio db, o corre una imagen vieja. Revisar settings.py y reconstruir.
`connection refused` a la base al arrancar	La API arrancó antes de que Postgres estuviera listo. Esperar unos segundos y ejecutar `docker restart discretizando-el-tiempo-continuo-api-1`.
`port is already allocated`	El puerto 8000 o 5432 está ocupado por otro programa. Cerrarlo o cambiar el puerto de la izquierda en docker-compose.yml.
`relation ... does not exist`	Faltan las tablas. Ejecutar el paso 4 (`migrate`).